LAPORAN PRAKTIKUM
MOBILE PROGRAMMING (E)
Jumat, 9 Oktober 2026

Desain GUI, Navigasi, dan Integrasi API dengan JSON Serialization di Flutter

Dosen Pengampu:
Khadijah Fahmi Hayati Holle, S.Kom., M.Kom

Nama : Muhammad Refki Andesta

NIM : 240605110232

PRODI TEKNIK INFORMATIKA

FAKULTAS SAINS DAN TEKNOLOGI

UIN MAULANA MALIK IBRAHIM MALANG

2026

A. Pendahuluan dan Tujuan Praktikum

Perkembangan teknologi mobile mendorong kebutuhan akan aplikasi yang memiliki antarmuka menarik, navigasi yang mudah digunakan, serta kemampuan untuk mengakses dan menampilkan data secara dinamis. Flutter merupakan framework yang dapat digunakan untuk mengembangkan aplikasi mobile dengan memanfaatkan bahasa pemrograman Dart dan berbagai widget untuk membangun antarmuka pengguna.

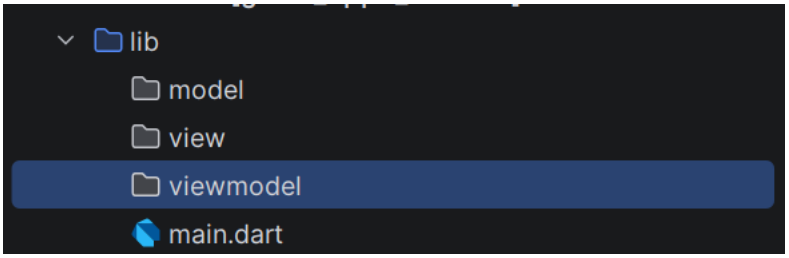
Pada praktikum ini, dilakukan pengembangan aplikasi mobile menggunakan Flutter dengan menerapkan desain Graphical User Interface (GUI), navigasi antarhalaman menggunakan named route, serta integrasi Application Programming Interface (API) untuk mengambil data dari sumber eksternal. Selain itu, digunakan JSON serialization dan deserialization untuk mengubah data JSON menjadi objek Dart sehingga data dapat diolah dan ditampilkan pada aplikasi dengan lebih terstruktur.

Melalui praktikum ini, dikembangkan aplikasi *Flutter Game Store* yang menampilkan daftar game beserta informasi detailnya. Proses pengambilan data dilakukan menggunakan paket http, sedangkan pengelolaan model data memanfaatkan json_serializable dan build_runner. Dengan demikian, praktikum ini diharapkan dapat meningkatkan pemahaman mengenai pengembangan aplikasi mobile yang interaktif, dinamis, dan terintegrasi dengan layanan API eksternal.

B. Langkah Praktikum dan Kode Program

1. Halaman Home

- **Di dalam folder proyek buat tiga folder**



- **Buka file pubspec.yaml, lalu tambahkan baris berikut di bawah bagian dependencies:**

```yaml
dependencies:
  flutter:
    sdk: flutter
  http: ^1.1.0
  json_annotation: ^4.9.0
  readmore: ^2.2.0

  # The following adds the Cupertino Icons
  # Use with the CupertinoIcons class for
  cupertino_icons: ^1.0.8

dev_dependencies:
  flutter_test:
    sdk: flutter
  build_runner: ^2.4.8
  json_serializable: ^6.7.1
```

Kode tersebut merupakan bagian dari pubspec.yaml yang berfungsi untuk mengatur package yang digunakan dalam aplikasi Flutter. http digunakan untuk mengambil data dari API, json_annotation dan json_serializable membantu mengolah data JSON, sedangkan readmore digunakan untuk membuat teks panjang memiliki fitur Read More. cupertino_icons menyediakan ikon bergaya iOS, sementara flutter_test dan build_runner digunakan untuk keperluan pengujian dan pembuatan kode saat proses pengembangan aplikasi.

- **Buka file main.dart di folder lib/, lalu isi dengan kode**

README.md | main.dart | pubspec.yaml

```dart
import 'package:flutter/material.dart';
import 'package:flutter/services.dart';
import 'package:game_app/view/home.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

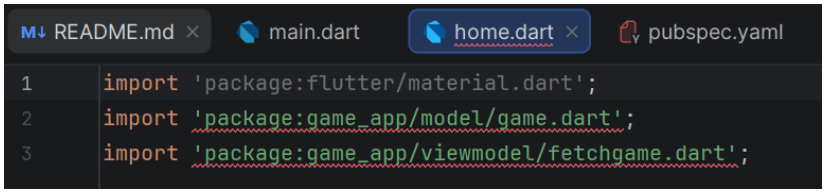
  @override
  Widget build(BuildContext context) {
    SystemChrome.setSystemUIOverlayStyle(
      const SystemUiOverlayStyle(
        statusBarColor: Colors.transparent,
        statusBarIconBrightness: Brightness.dark,
      ),
    );

    return const MaterialApp(
      debugShowCheckedModeBanner: false,
      title: 'Flutter Game Store',
      home: Home(),
    );
```

Kode tersebut merupakan program utama aplikasi Flutter. main() menjalankan aplikasi dengan memanggil MyApp(). Class MyApp mengatur tampilan dan konfigurasi awal

aplikasi, seperti menghilangkan tulisan debug, memberi nama aplikasi Flutter Game Store, serta menentukan halaman Home() sebagai halaman pertama. SystemChrome.setSystemUIOverlayStyle() digunakan untuk mengatur tampilan status bar agar transparan dengan ikon berwarna gelap.

- **Import pada home.dart**



Kode tersebut digunakan untuk mengimpor library yang dibutuhkan. material.dart menyediakan komponen tampilan Flutter, game.dart digunakan untuk mengakses model atau struktur data game, sedangkan fetchgame.dart digunakan untuk mengambil data game dari API.

- **Kode class homestate home.dart**



Kode di atas merupakan class _HomeState yang berfungsi mengatur state atau kondisi halaman Home, termasuk proses mengambil dan menyimpan data game dari API ke dalam gameList._

- **Kode home.dart**

```
Widget build(BuildContext context) {
  return Scaffold(
    backgroundColor: Colors.amberAccent.shade400,
    body: SafeArea(
      child: Container(
        padding: const EdgeInsets.symmetric(
          horizontal: 10,
          vertical: 5,
        ), EdgeInsets.symmetric
        child: Column(
          children: [
            _searchBar(),
            const SizedBox(
              height: 10,
            ), SizedBox
            FutureBuilder<List<Game>>(
              future: fetchGames(),
              builder: (context, snapshot) {
                if (snapshot.connectionState ==
                    ConnectionState.waiting) {
                  return const Expanded(
                    child: Center(
                      child: CircularProgressIndicator(),
                    ), Center
                  ); Expanded
```

```
} else if (snapshot.hasError) {
  return Text('Error: ${snapshot.error}');
} else if (!snapshot.hasData ||
    snapshot.data!.isEmpty) {
  return const Text('Tidak ada data game.');
} else {
  final games = snapshot.data!.take(25).toList();

  return Expanded(
    child: ListView.builder(
      itemCount: games.length,
      itemBuilder: (context, index) {
        final game = games[index];

        return GestureDetector(
          onTap: () {
            Navigator.pushNamed(
              context,
              '/detail',
              arguments: game.id,
            );
          },
          child: _listItem(
            game.thumbnail,
            game.title,
```



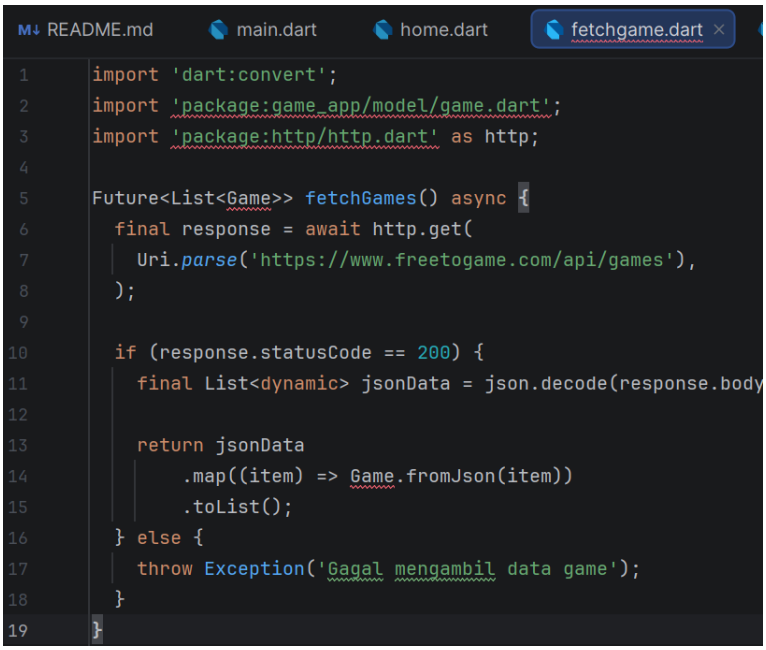
```
TextField _searchBar() {
  return TextField(
    cursorColor: Colors.blue,
    decoration: InputDecoration(
      fillColor: Colors.blue.shade50,
      filled: true,
      border: OutlineInputBorder(
        borderRadius: BorderRadius.circular(20),
        borderSide: const BorderSide(
          width: 0,
          style: BorderStyle.none,
        ), BorderSide
      ), OutlineInputBorder
      contentPadding: const EdgeInsets.symmetric(
        vertical: 0,
        horizontal: 20,
      ), EdgeInsets.symmetric
      prefixIcon: const Icon(
        Icons.search_outlined,
        color: Colors.blue,
        size: 30,
      ), Icon
      hintText: 'Cari game',
      hintStyle: TextStyle(
        fontSize: 14,
        color: Colors.grey.withOpacity(0.7),
      ), TextStyle
    ), InputDecoration
  ); TextField
}

Card _listItem(
    String urlCover,
    String judul,
    String genre,
    ) {...}
```

Kode di atas merupakan bagian build() yang digunakan untuk membuat tampilan halaman Home, seperti search bar dan daftar game. FutureBuilder digunakan untuk menampilkan data game dari API, menampilkan loading saat data sedang dimuat, pesan error jika terjadi kesalahan, dan daftar game jika data berhasil diperoleh. Selain itu, _searchBar() digunakan untuk membuat kolom pencarian, sedangkan _listItem() digunakan untuk membuat tampilan setiap game dalam bentuk Card yang berisi

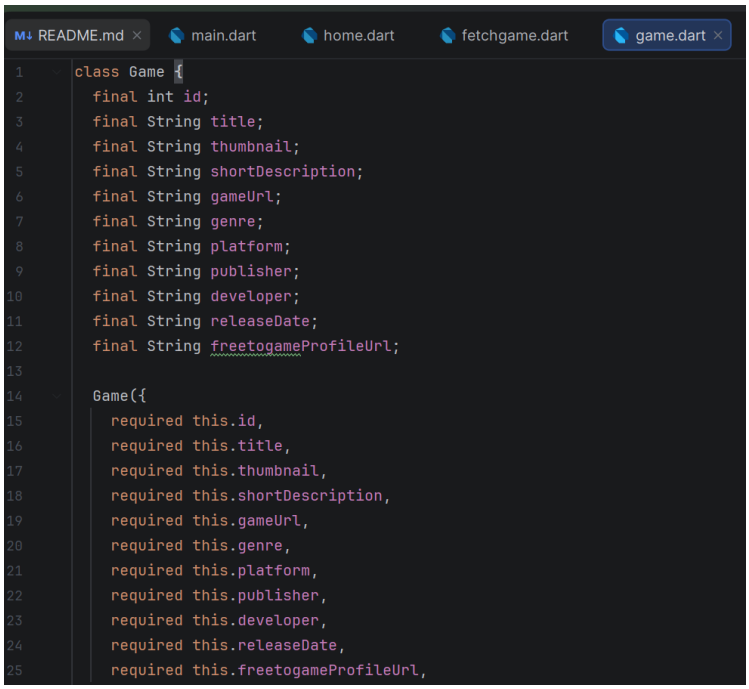
gambar, judul, dan genre game.

- **Kode fatchgame.dart**



Kode tersebut merupakan fungsi fetchGames() yang digunakan untuk mengambil data game dari API FreeToGame. Data yang diterima dalam format JSON kemudian diubah menjadi objek Game dan disimpan dalam bentuk List<Game>. Jika proses pengambilan data berhasil, daftar game dikembalikan, sedangkan jika gagal akan menampilkan pesan “Gagal mengambil data game”.

- **Kode game.dart**

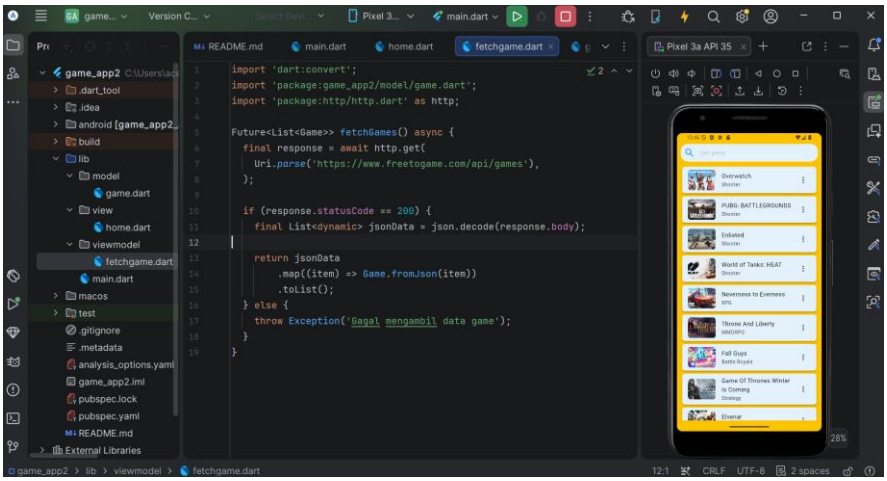


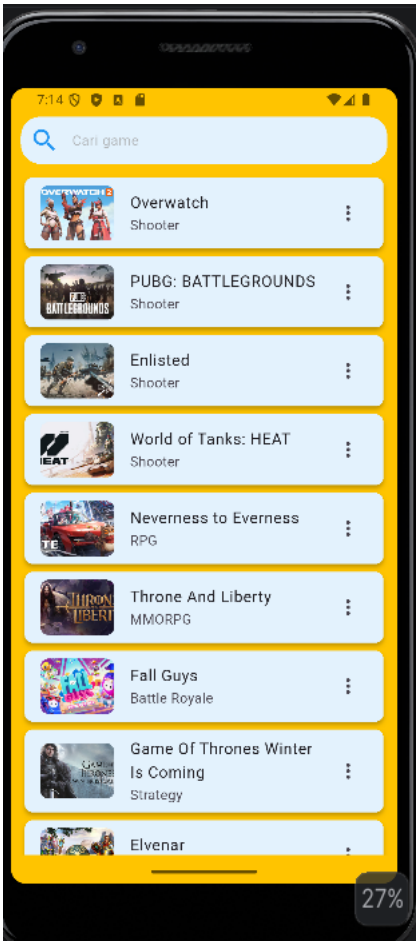
```dart
    required this.freetogameProfileUrl,
  });

  factory Game.fromJson(Map<String, dynamic> json) {
    return Game(
      id: json['id'],
      title: json['title'],
      thumbnail: json['thumbnail'],
      shortDescription: json['short_description'],
      gameUrl: json['game_url'],
      genre: json['genre'],
      platform: json['platform'],
      publisher: json['publisher'],
      developer: json['developer'],
      releaseDate: json['release_date'],
      freetogameProfileUrl: json['freetogame_profile_url'],
    );
  }
}
```

Kode tersebut merupakan model Game yang digunakan untuk menampung data game. Class ini mendefinisikan berbagai informasi game seperti ID, judul, gambar, genre, platform, publisher, dan developer. Game.fromJson() digunakan untuk mengubah data JSON dari API menjadi objek Game sehingga data dapat digunakan dengan lebih mudah di dalam aplikasi.

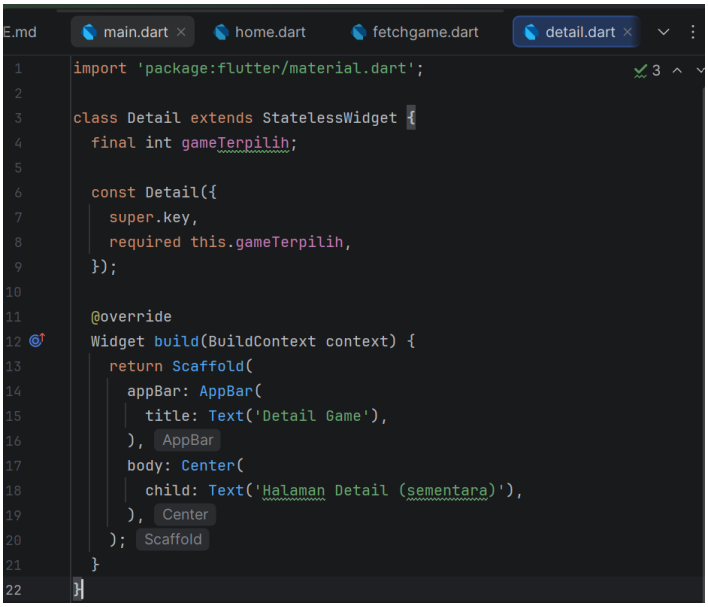
- **Hasil Tampilan Aplikasi (1)**





2. Bagian Detail Game

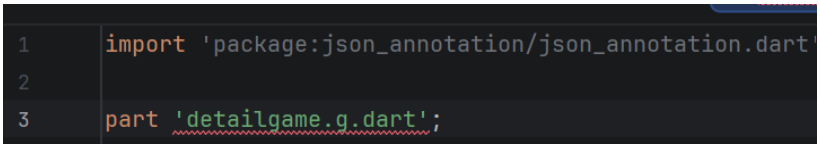
- **Tambahkan konstruktor untuk menerima parameter gameTerpilih berupa int (id game yang dikirim dari Home). Di method build() buat Scaffold sederhana dengan AppBar dan body berisi placeholder teks seperti “Halaman Detail (sementara)”.**



Kode tersebut merupakan class Detail yang digunakan untuk membuat halaman detail

game. Variabel gameTerpilih menyimpan ID game yang dipilih dari halaman Home. Saat ini, halaman tersebut masih berupa tampilan sementara dengan AppBar bertuliskan “Detail Game” dan teks “Halaman Detail (sementara)” di bagian tengah.

- **File detailgame.dart import**

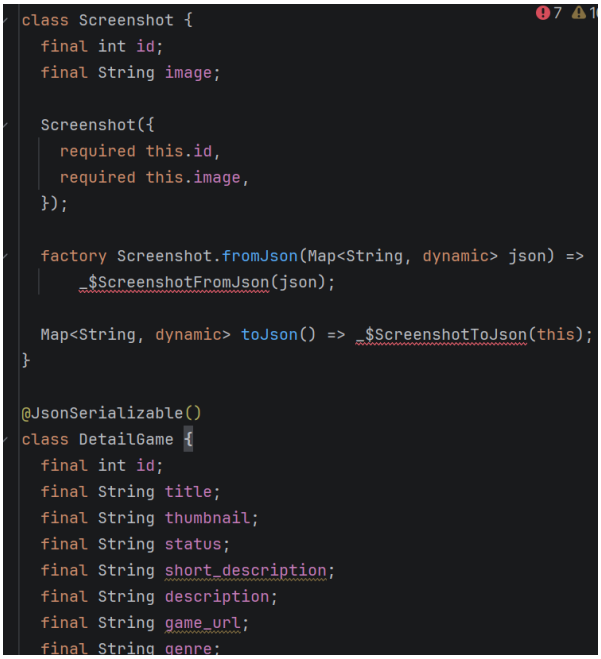


Kode tersebut digunakan untuk mempersiapkan proses otomatisasi JSON pada model DetailGame. json_annotation menyediakan anotasi untuk proses konversi JSON, sedangkan part 'detailgame.g.dart' menghubungkan file dengan kode hasil generate dari json_serializable.

- **class MinimumSystemRequirements**



- **class Screenshot dan Class Detail game**



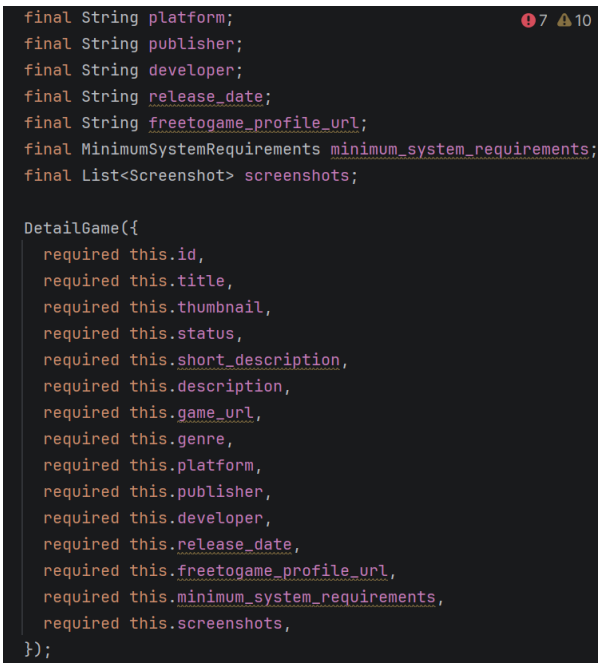
```dart
class Screenshot {
  final int id;
  final String image;

  Screenshot({
    required this.id,
    required this.image,
  });

  factory Screenshot.fromJson(Map<String, dynamic> json) =>
      _$ScreenshotFromJson(json);

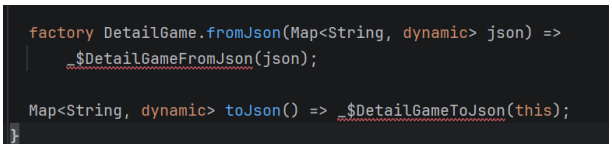
  Map<String, dynamic> toJson() => _$ScreenshotToJson(this);
}

@JsonSerializable()
class DetailGame {
  final int id;
  final String title;
  final String thumbnail;
  final String status;
  final String short_description;
  final String description;
  final String game_url;
  final String genre;
```



```dart
final String platform;
final String publisher;
final String developer;
final String release_date;
final String freetogame_profile_url;
final MinimumSystemRequirements minimum_system_requirements;
final List<Screenshot> screenshots;

DetailGame({
  required this.id,
  required this.title,
  required this.thumbnail,
  required this.status,
  required this.short_description,
  required this.description,
  required this.game_url,
  required this.genre,
  required this.platform,
  required this.publisher,
  required this.developer,
  required this.release_date,
  required this.freetogame_profile_url,
  required this.minimum_system_requirements,
  required this.screenshots,
});
```



```dart
factory DetailGame.fromJson(Map<String, dynamic> json) =>
    _$DetailGameFromJson(json);

Map<String, dynamic> toJson() => _$DetailGameToJson(this);
}
```

Kode tersebut membuat dua **model Dart**, yaitu Screenshot untuk menyimpan data screenshot game berupa id dan image, serta DetailGame untuk menyimpan informasi lengkap sebuah game seperti judul, gambar, deskripsi, genre, platform, publisher, developer, dan tanggal rilis. minimum_system_requirements digunakan untuk menyimpan spesifikasi minimum game, sedangkan screenshots berupa List<Screenshot> untuk menyimpan beberapa screenshot. fromJson() digunakan

untuk mengubah data JSON dari API menjadi objek Dart, dan toJson() untuk mengubah objek kembali menjadi JSON. Annotation @JsonSerializable() membuat proses tersebut dapat digenerate secara otomatis.

- **fetchgame.dart di folder viewmodel/ kemudian tambahkan kode program**

```dart
Future<Map<String, dynamic>> fetchDataFromAPI(int id) async {
  final response = await http.get(
    Uri.parse('https://www.freetogame.com/api/game?id=$id'),
  );

  if (response.statusCode == 200) {
    return json.decode(response.body);
  } else {
    throw Exception('Gagal mengambil detail game');
  }
}
```

Kode tersebut digunakan untuk mengambil data detail game dari API FreeToGame berdasarkan id game. Fungsi fetchDataFromAPI() melakukan request GET ke API menggunakan http.get(). Jika statusCode bernilai 200, berarti data berhasil diambil dan response.body diubah dari JSON menjadi Map<String, dynamic> menggunakan json.decode(). Jika request gagal, program akan menjalankan throw Exception() dengan pesan bahwa detail game gagal diambil.

- **Perbarui Kode detail.dart**

```dart
import 'package:flutter/material.dart';
import 'package:game_app/viewmodel/fetchgame.dart';
import 'package:game_app/model/detailgame.dart';
import 'package:readmore/readmore.dart';

class Detail extends StatelessWidget {
  final int gameTerpilih;

  const Detail({super.key, required this.gameTerpilih});

  Future<DetailGame> fetchData() async {
    final jsonData = await fetchDataFromAPI(gameTerpilih);
    return DetailGame.fromJson(jsonData);
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      backgroundColor: Colors.amberAccent.shade400,
      body: FutureBuilder(
        future: fetchData(),
        builder: (context, snapshot) {
          if (snapshot.connectionState == ConnectionState.waiting) {
            return const Center(
              child: CircularProgressIndicator(),
```

```
      ); Center
    } else if (snapshot.hasError) {
      return Center(
        child: Text(
          'Terjadi kesalahan: ${snapshot.error}',
        ), Text
      ); Center
    } else if (!snapshot.hasData) {
      return const Center(
        child: Text('Data tidak tersedia'),
      ); Center
    } else {
      final game = snapshot.data!;

      return Column(
        children: [
          SizedBox(
            width: double.infinity,
            child: Stack(
              children: [
                ClipRRect(
                  borderRadius: const BorderRadius.only(
                    bottomLeft: Radius.circular(25),
                    bottomRight: Radius.circular(25),
```

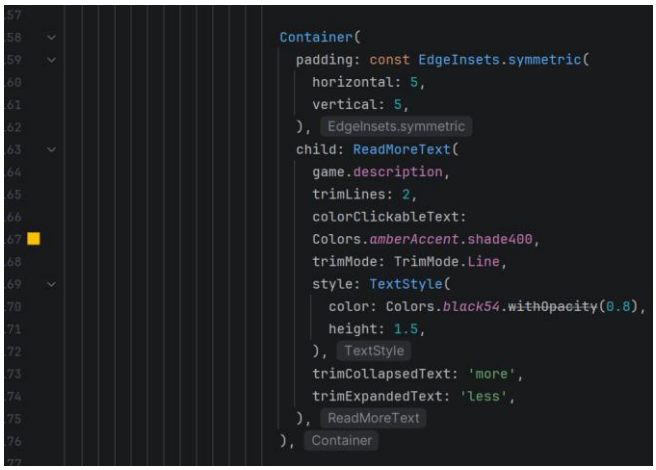
```
                ), BorderRadius.only
                child: Image.network(
                  game.thumbnail,
                  fit: BoxFit.cover,
                  height: 300,
                ), Image.network
              ), ClipRRect
              Positioned(
                top: 25,
                left: 10,
                child: GestureDetector(
                  onTap: () {
                    Navigator.pop(context);
                  },
                  child: Container(
                    padding: const EdgeInsets.all(5),
                    decoration: BoxDecoration(
                      color: Colors.grey.withOpacity(0.8),
                      shape: BoxShape.circle,
                    ), BoxDecoration
                    child: const Icon(
                      Icons.arrow_back_ios_outlined,
                      color: Colors.white,
                    ), Icon
                  ), Container
                ), GestureDetector
```

```
      const SizedBox(height: 10),                    ⚠6 ✓9

      Expanded(
        child: Container(
          padding: const EdgeInsets.symmetric(
            horizontal: 10,
            vertical: 10,
          ), EdgeInsets.symmetric
          width: double.infinity,
          decoration: BoxDecoration(
            color: Colors.blue.shade50,
            borderRadius: BorderRadius.circular(10),
          ), BoxDecoration
          child: SingleChildScrollView(
            child: Column(
              children: [
                SizedBox(
                  width: double.maxFinite,
                  child: Text(
                    'Minimum System Requirements',
                    style: TextStyle(
                      fontSize: 14,
                      fontWeight: FontWeight.bold,
                      color: Colors.black54.withOpacity(0.8),
                    ), TextStyle
```



```
Column(
  children: [
    _requirementRow(
      'OS',
      game.minimum_system_requirements.os,
    ),
    _requirementRow(
      'Processor',
      game.minimum_system_requirements.processo
    ),
    _requirementRow(
      'Memory',
      game.minimum_system_requirements.memory,
    ),
    _requirementRow(
      'Graphics',
      game.minimum_system_requirements.graphics
    ),
    _requirementRow(
      'Storage',
      game.minimum_system_requirements.storage,
    ),
  ],
```



```
Container(
  padding: const EdgeInsets.symmetric(
    horizontal: 5,
    vertical: 5,
  ), EdgeInsets.symmetric
  child: ReadMoreText(
    game.description,
    trimLines: 2,
    colorClickableText:
    Colors.amberAccent.shade400,
    trimMode: TrimMode.Line,
    style: TextStyle(
      color: Colors.black54.withOpacity(0.8),
      height: 1.5,
    ), TextStyle
    trimCollapsedText: 'more',
    trimExpandedText: 'less',
  ), ReadMoreText
), Container
```

Kode tersebut digunakan untuk membuat halaman detail game. Class Detail menerima gameTerpilih sebagai ID game, kemudian mengambil data dari API melalui fetchData() dan mengubahnya menjadi objek DetailGame. FutureBuilder digunakan untuk mengatur kondisi saat data masih dimuat, terjadi error, atau data berhasil diperoleh. Setelah berhasil, halaman menampilkan thumbnail game, tombol kembali, minimum system requirements, beberapa screenshot game, dan deskripsi game yang dapat diperpanjang menggunakan ReadMoreText. Fungsi _requirementRow() digunakan untuk membuat baris informasi spesifikasi, sedangkan _itemList() digunakan untuk menampilkan gambar screenshot game.

- **Buka file main.dart kemudian import file detail.dart kemudian edit kode program**

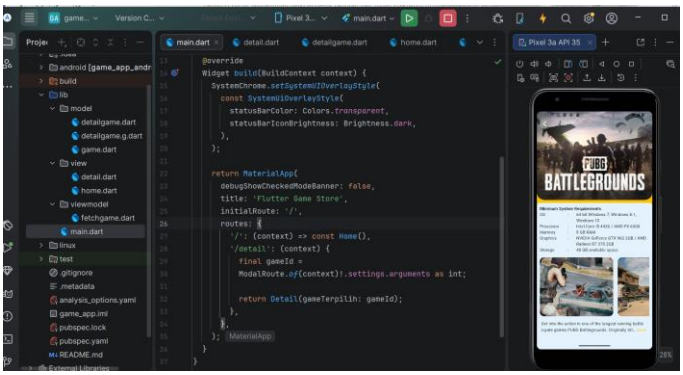
```
import 'package:flutter/material.dart';
import 'package:flutter/services.dart';
import 'package:game_app/view/home.dart';
import 'package:game_app/view/detail.dart';
```

```
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      title: 'Flutter Game Store',
      initialRoute: '/',
      routes: {
        '/': (context) => const Home(),
        '/detail': (context) {
          final gameId =
          ModalRoute.of(context)!.settings.arguments as int;

          return Detail(gameTerpilih: gameId);
        },
      },
    );  MaterialApp
  }
}
```

Kode tersebut digunakan untuk mengatur navigasi atau perpindahan halaman pada aplikasi Flutter. MaterialApp menjadi konfigurasi utama aplikasi, sedangkan initialRoute: '/' menentukan bahwa halaman pertama yang dibuka adalah Home. Pada routes, route '/' mengarah ke halaman Home, sementara '/detail' mengarah ke halaman Detail. ID game diambil melalui ModalRoute.of(context)!.settings.arguments, kemudian digunakan sebagai gameTerpilih untuk menampilkan detail game yang dipilih.

- **Hasil Tampilan Aplikasi (2)**





C. Kesimpulan dan Ringkasan Hasil

Berdasarkan praktikum Desain GUI, Navigasi, dan Integrasi API dengan JSON Serialization di Flutter, diperoleh beberapa pengetahuan sebagai berikut:

- Penggunaan widget Flutter seperti ListView, Card, dan TextField mempermudah pembuatan antarmuka aplikasi yang interaktif, terstruktur, dan mudah digunakan.
- Penerapan named route membantu mengatur navigasi antarhalaman secara terstruktur melalui konfigurasi route yang terpusat pada MaterialApp.
- Penggunaan Navigator.pushNamed() untuk membuka halaman detail dan Navigator.pop() untuk kembali ke halaman sebelumnya mempermudah proses navigasi dalam aplikasi.
- JSON serialization dan deserialization memungkinkan proses konversi data antara format JSON dan objek Dart sehingga data lebih mudah dikelola.
- Integrasi API menggunakan paket http memungkinkan aplikasi mengambil data game dari sumber eksternal dan menampilkannya secara dinamis.
- Penggunaan FutureBuilder membantu menangani proses pengambilan data asynchronous, termasuk kondisi loading, error, dan keberhasilan memperoleh data.
- Pemisahan kode ke dalam folder model, view, dan viewmodel membuat struktur

program lebih modular, terorganisasi, dan mudah dikembangkan.

- Penggunaan json_serializable dan build_runner membantu menghasilkan kode konversi JSON secara otomatis sehingga mengurangi penulisan kode secara manual.

Dapat disimpulkan bahwa aplikasi Flutter dapat digunakan untuk membuat aplikasi game store yang memiliki tampilan GUI, navigasi antarhalaman, serta integrasi dengan API. Data game dari API FreeToGame berhasil diambil dan diolah menggunakan JSON Serialization sehingga dapat ditampilkan dalam bentuk daftar game dan halaman detail. Penggunaan `FutureBuilder` membantu menangani proses pemuatan dan pengambilan data secara asynchronous, sedangkan `MaterialApp` dan `routes` digunakan untuk mengatur navigasi dari halaman Home ke halaman Detail. Selain itu, model `Game`, `DetailGame`, `Screenshot`, dan `MinimumSystemRequirements` mempermudah pengelolaan data JSON menjadi objek Dart. Dengan praktikum ini, dapat dipahami penerapan dasar desain GUI, navigasi, pengambilan data API, serta pengolahan JSON pada aplikasi mobile menggunakan Flutter.